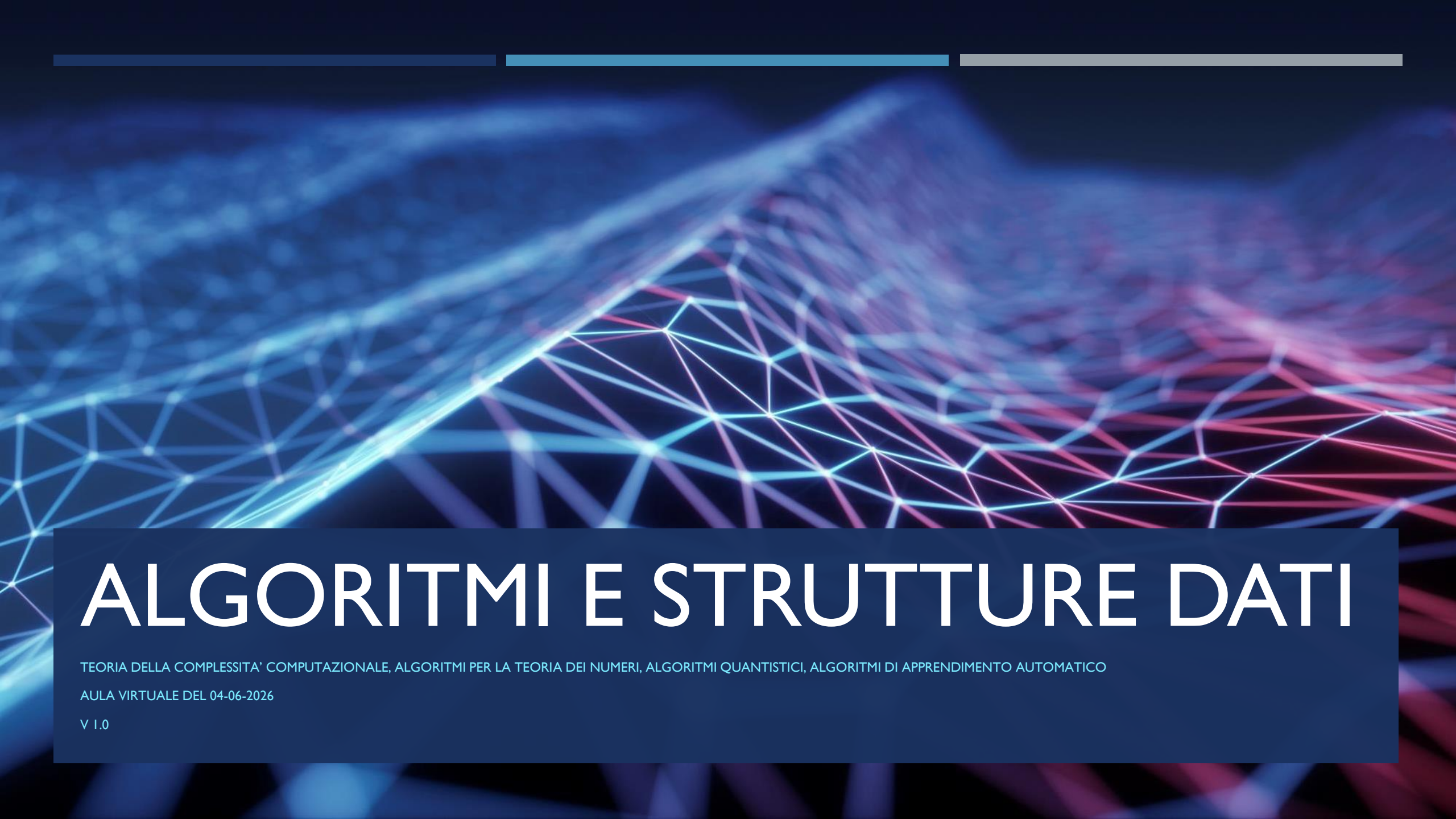


ALGORITMI E STRUTTURE DATI

TEORIA DELLA COMPLESSITA' COMPUTAZIONALE, ALGORITMI PER LA TEORIA DEI NUMERI, ALGORITMI QUANTISTICI, ALGORITMI DI APPRENDIMENTO AUTOMATICO

AULA VIRTUALE DEL 04-06-2026

V 1.0

TEORIA DELLA NP-COMPLETEZZA: INTRODUZIONE

I problemi finora rappresentati ammettono algoritmi risolutivi il cui tempo di esecuzione è polinomiale nella dimensione n dell'istanza di ingresso (o dati in input).
Ad esempio, abbiamo visto algoritmi che richiedono tempo lineare o quadratico. Anche tempi di esecuzione in cui compaiono fattori logaritmici rientrano in questa classificazione, poiché possono essere limitati superiormente da un polinomio in n: ad esempio, O(n log n) = O($n^{1+\varepsilon}$) per ogni $\varepsilon > 0$.

La classe dei problemi che ammettono **algoritmi polinomiali è nota come classe P,** e questi problemi sono ritenuti trattabili.

Per molti problemi di grande interesse pratico algoritmi polinomiali non solo non sono stati ancora progettati, ma si presume che non esistano affatto.

TEORIA DELLA NP-COMPLETEZZA: INTRODUZIONE

Per molti di questi problemi, gli unici algoritmi risolutivi attualmente noti richiedono tempo almeno esponenziale in n. Ciò può essere proibitivo per un loro effettivo impiego su istanze reali di grandi dimensioni: per avere una soluzione, dovremmo attendere mesi, anni, o magari secoli, anche usando i calcolatori più potenti e veloci disponibili sul mercato!

Nella teoria della complessità computazionale desideriamo caratterizzare i problemi in termini delle risorse di calcolo richieste per la soluzione, quali tempo ed occupazione di memoria: informalmente, una classe di complessità è un insieme di problemi che possono essere risolti usando le stesse risorse di calcolo.

TEORIA DELLA NP-COMPLETEZZA: INTRODUZIONE

I problemi algoritmici possono avere una natura molto **diversa**, tentiamo innanzitutto di darne almeno una prima caratterizzazione.

In generale, possiamo pensare a un problema P come a una relazione $P \subseteq I \times S$, dove I è l'insieme delle istanze di ingresso e S quello delle soluzioni. Possiamo anche immaginare di avere un predicato che, presa un'istanza $x \in I$ ed una soluzione $x \in S$, restituisce 1 se $(x,s) \in P$ (ovvero se s è una soluzione del problema P sull'istanza x), e 0 altrimenti.

Basandoci su questa terminologia, classifichiamo innanzitutto i problemi in problemi di **decisione**, di **ricerca** o di **ottimizzazione**.

Per classificare un problema non basta dire "sembra difficile"; bisogna dimostrare formalmente in quale classe di complessità si trova e come si collega ad altri problemi già noti.

TEORIA DELLA NP-COMPLETEZZA: INTRODUZIONE

Dati un problema di decisione P ed un algoritmo A, diciamo che A risolve il problema P se A restituisce 1 su un'istanza x se e solo se (x,1) ∈ P. Inoltre, diciamo che A risolve P in tempo t(n) e spazio s(n) se il tempo di esecuzione e l'occupazione di memoria di A sono rispettivamente t(n) e s(n). Possiamo definire alcune importanti classi di complessità.

Definizione 1: Data una qualunque funzione f(n), chiamiamo TIME(f(n)) e SPACE(f(n)) gli insiemi dei problemi decisionali che possono essere risolti rispettivamente in tempo e spazio O(f(n)).

Definizione 2: La classe P è la classe dei problemi risolvibili in tempo polinomiale nella dimensione n dell'istanza di ingresso:

$P = \cup_{c=0}^{\infty} \text{TIME}\ (n^c)$

TEORIA DELLA NP-COMPLETEZZA: INTRODUZIONE

La classe PSpace è la classe dei problemi risolvibili in spazio polinomiale nella dimensione n dell'istanza di ingresso:

PSpace = $\cup_{c=0}^{\infty}$ Space (n^c)

La classe ExpTime è la classe dei problemi risolvibili in tempo esponenziale nella dimensione n dell'istanza di ingresso:

ExpTime = $\cup_{c=0}^{\infty}$ Time ($2^{n\hat{}c}$)

TEORIA DELLA NP-COMPLETEZZA: INTRODUZIONE

Nell'ambito dei **problemi di decisione** siamo interessati a verificare se una istanza x del problema soddisfa una certa proprietà. Spesso, in caso di risposta affermativa, oltra alla semplice risposta binaria si richiede anche di fornire un qualche oggetto y, dipendente dall'istanza x e dal problema specifico, che possa certificare il fatto che x soddisfa la proprietà, giustificando quindi la risposta affermativa.

Tale oggetto è noto come certificato, ed intuitivamente è una soluzione costruttiva per il problema.

Quindi, NP è la classe dei problemi decisionali che ammettono certificati verificabili in tempo polinomiale.

Quando parliamo di P, NP, NP-completezza e riduzioni, di solito lavoriamo con problemi decisionali.

Un problema decisionale è un problema la cui risposta è:

sì oppure **no**

TEORIA DELLA NP-COMPLETEZZA: INTRODUZIONE

Esempi:

Problema	Versione decisionale
Trovare il cammino minimo	Esiste un cammino di costo al massimo k?
Trovare una clique massima	Esiste una clique di dimensione almeno k?
Trovare un ciclo hamiltoniano	Esiste un ciclo che visita ogni vertice una sola volta?
TSP	Esiste un tour di costo al massimo B?

TEORIA DELLA NP-COMPLETEZZA: INTRODUZIONE

Definizione 3: data una qualunque funzione f(n), chiamiamo NTime(f(n)) l'insieme dei problemi decisionali che possono essere risolti da un algoritmo non deterministico in tempo O(f(n)). La classe NP è la classe dei problemi risolvibili in tempo polinomiale non deterministico nella dimensione n dell'istanza di ingresso:

NP = $\cup_{c=0}^{\infty}$ NTime (n^c)

TEORIA DELLA NP-COMPLETEZZA: INTRODUZIONE

Problemi NP-completi: se esistesse un algoritmo polinomiale per risolvere uno solo di questi problemi, allora tutti i problemi in NP potrebbero essere risolti in tempo polinomiale e l'annosa questione P =N P. In altri termini o i problemi Np-completi sono tutti risolvibili deterministicamente in tempo polinomiale, o nessuno di essi lo è.

Definizione 4 (Riducibilità polinomiale): Dati due problemi decisionali $P_1 \subseteq I_1 \times \{0,1\}$ e $P_2 \subseteq I_2 \times \{0,1\}$, diremo che P_1 è riducibile polinomialmente a P_2 (e scriveremo $P_1 \prec P_2$) se esiste una funzione f: $I_1 \longrightarrow I_2$ con le seguenti proprietà:

- f è calcolabile in tempo polinomiale
- per ogni istanza x del problema P_1 ed ogni soluzione s $\in \{0,1\}$, (x,s) $\in P_1$ se e solo se (f(x),s) $\in P_2$.

TEORIA DELLA NP-COMPLETEZZA: INTRODUZIONE

L'NP-completezza va pensata come uno strumento che permette di confrontare le classi P e NP limitandosi a studiare la difficoltà di singoli problemi.

Teorema di Cook: Il problema della soddisfacibilità di espressioni Booleane in forma normale congiuntiva è NP-completo.

Osservazione: il teorema di Cook ha mostrato un algoritmo che, dati un qualunque problema P (che vogliamo ridurre a soddisfacibilità) ed una qualunque istanza x per P, costruisce una espressione Booleana in forma normale congiuntiva che descrive il calcolo di un algoritmo non deterministico per risolvere P su x.

TEORIA DELLA NP-COMPLETEZZA: INTRODUZIONE

Quindi, che cosa significa che un problema appartiene a NP?

Un problema A appartiene a NP se, per ogni istanza con risposta “sì”, esiste un certificato verificabile in tempo polinomiale.
In altre parole: $A \in NP$ se esiste un algoritmo verificatore V tale che:

dato un input x; dato un certificato c; V(x,c) controlla in tempo polinomiale se c dimostra che x è una istanza positiva.

La cosa importante è che non chiediamo di trovare efficientemente il certificato. Chiediamo solo di poterlo verificare efficientemente.

TEORIA DELLA NP-COMPLETEZZA: INTRODUZIONE

Il certificato è una soluzione candidata.

Il verificatore è un algoritmo deterministico che controlla se il certificato è valido.

Schema:

istanza x + certificato c ⟶ verifica in tempo polinomiale

TEORIA DELLA NP-COMPLETEZZA: INTRODUZIONE

Che cos'è una riduzione polinomiale?

Una riduzione polinomiale serve a confrontare la difficoltà di due problemi.

Supponiamo di avere due problemi decisionali A e B.

Diciamo che: $A \leq_p B$
se esiste una funzione f, calcolabile in tempo polinomiale, tale che per ogni istanza x di A:
$x \in A \iff f(x) \in B$

Questa formula è fondamentale:
L'istanza x ha risposta sì per A se e solo se l'istanza trasformata f(x) ha risposta sì per B.
La funzione f trasforma istanze di A in istanze di B, preservando la risposta.

TEORIA DELLA NP-COMPLETEZZA: INTRODUZIONE

Quando scriviamo: $A \leq_p B$

se sapessi risolvere efficientemente B, allora saprei risolvere efficientemente anche A.
Infatti, per risolvere A: prendo un'istanza x di A;

calcolo f(x), istanza di B; risolvo B su f(x); restituisco la stessa risposta.

$$x \xrightarrow{f} f(x)$$

$$A(x) = \text{sì} \iff B(f(x)) = \text{sì}$$

Se f è polinomiale e B è risolvibile in tempo polinomiale, allora anche A è risolvibile in tempo polinomiale.
Quindi:
$A \leq_p B$ e $B \in P \Rightarrow A \in PA$

TEORIA DELLA NP-COMPLETEZZA: INTRODUZIONE

Differenza tra “appartiene a NP” e “NP-hard”

Questi due concetti sono diversi.
Dire che:
$B \in NP$

significa:

se qualcuno mi dà una soluzione candidata, posso verificarla in tempo polinomiale.

Dire che: B e` NP-hard
significa: B è almeno difficile quanto tutti i problemi in NP.

Un problema può essere:

TEORIA DELLA NP-COMPLETEZZA: INTRODUZIONE

Caso	Significato
In NP ma non noto NP-hard	verificabile efficientemente, ma non necessariamente tra i più difficili
NP-hard ma non in NP	almeno difficile quanto NP, ma forse non verificabile con certificato polinomiale
NP-completo	sia in NP sia NP-hard
In P	risolvibile efficientemente, quindi anche in NP

TEORIA DELLA NP-COMPLETEZZA: ALGORITMI DI APPROSSIMAZIONE

Cosa fare nel caso si renda necessario risolvere un problema di ottimizzazione la cui versione decisionale sia NP-completa?

Sperare di poter trovare un algoritmo polinomiale non sembra ragionevole, poiché l'evidenza suggerisce che $P \neq NP$. A volte, avere una soluzione esatta non è però strettamente necessario: una soluzione che non si discosti troppo da quella ottimale potrebbe comunque risultare utile, e magari potremmo anche sperare che sia più facile da calcolare. La teoria dell'approssimazione formalizza questa semplice idea.

Dato un problema di ottimizzazione, esistono in generale **numerose soluzioni ammissibili** per quel problema, ma tipicamente **non tutte sono ottime**. Ad esempio, consideriamo il problema della colorazione di un grafo: una soluzione ammissibile consiste nell'assegnare un colore diverso ad ogni vertice, usando pertanto n colori. È facile vedere che questa soluzione è ottima se e solo se il grafo è completo: se infatti mancasse almeno un arco, diciamo (u,v), i vertici u e v potrebbero essere assegnati con lo stesso colore, usando pertanto meno di n colori. Vediamo innanzitutto come misurare la distanza di una qualunque soluzione ammissibile da una soluzione ottima.

TEORIA DELLA NP-COMPLETEZZA: ALGORITMI DI APPROSSIMAZIONE

Definizione 5: Sia P un problema di ottimizzazione. Diciamo che un algoritmo di approssimazione per P ha un fattore di approssimazione p(n) se il costo C della soluzione prodotta dall'algoritmo su una qualunque istanza di dimensione n differisce di un fattore moltiplicativo <=p(n) dal costo C* di una soluzione ottima:

$\max\{C/C^*, C'/C\} <= p(n)$

ALGORITMI PER LA TEORIA DEI NUMERI: CONCETTI ELEMENTARI

Algoritmi per la teoria dei numeri sono algoritmi usati per risolvere problemi di natura matematica, come calcolo di numeri primi, approssimazione di soluzioni, ottimizzazione, algebra lineare e analisi numerica. Sono importanti perché mostrano bene concetti fondamentali come:

- complessità computazionale;
- uso efficiente della memoria;
- iterazione e ricorrenza;
- approssimazione;
- convergenza;
- scelta delle strutture dati.

In letteratura due algoritmi classici sono il **Crivello di Eratostene** e il **Metodo del gradiente**.

ALGORITMI PER LA TEORIA DEI NUMERI: CRIVELLO DI ERATORESTENE

Il Crivello di Eratostene è un algoritmo efficiente per trovare tutti i numeri primi minori o uguali a un certo intero n. Un numero primo è un numero naturale maggiore di 1 divisibile solo per 1 e per sé stesso.

L'idea è eliminare progressivamente i multipli dei numeri primi.
Si parte assumendo che tutti i numeri da 2 a n siano primi. Poi:

- si prende il primo numero non eliminato, cioè 2;
- si eliminano tutti i suoi multipli: 4, 6, 8, 10, ...;
- si passa al successivo numero non eliminato, cioè 3;
- si eliminano i multipli di 3: 6, 9, 12, 15, ...;
- si continua fino a $\sqrt{n}$.

Alla fine, i numeri non eliminati sono primi.

ALGORITMI PER LA TEORIA DEI NUMERI: CRIVELLO DI ERATORESTENE

Complessità
La complessità temporale del Crivello di Eratostene è:

O(n log log n)

La complessità spaziale è:
O(n)

perché si usa un array booleano di dimensione n + 1. Alla fine, i numeri non eliminati sono primi.

ALGORITMI PER LA TEORIA DEI NUMERI: CRIVELLO DI ERATORESTENE

```
def crivello_eratostene(n):
    if n < 2:
        return []

    primo = [True] * (n + 1)
    primo[0] = False
    primo[1] = False

    p = 2
    while p * p <= n:
        if primo[p]:
            for multiplo in range(p * p, n + 1, p):
                primo[multiplo] = False
        p += 1

    return [i for i in range(n + 1) if primo[i]]


print(crivello_eratostene(30))
```

ALGORITMI PER LA TEORIA DEI NUMERI: METODO DEL GRADIENTE

Il Metodo del gradiente, o Gradient Descent, è un algoritmo numerico iterativo usato per trovare il minimo di una funzione.
È molto usato in:

- ottimizzazione;
- machine learning;
- reti neurali;
- regressione;
- problemi di minimo costo.

ALGORITMI PER LA TEORIA DEI NUMERI: METODO DEL GRADIENTE

Dato un problema del tipo: minimizzare f(x)

vogliamo trovare un valore x per cui la funzione f(x) assume il valore minimo.

Nel caso più semplice, x è una variabile reale. Nei casi più generali, x è un vettore:
x = (x1, x2, ..., xn)

ALGORITMI PER LA TEORIA DEI NUMERI: METODO DEL GRADIENTE

Il gradiente indica la direzione di massima crescita della funzione.

Quindi, per diminuire la funzione, bisogna muoversi nella direzione opposta al gradiente.

La regola di aggiornamento è:
$x_{k+1} = x_k - \alpha \nabla f(x_k)$
dove:
x_k è il punto corrente;
x_{k+1} è il punto successivo;
α è il passo di apprendimento, o learning rate;
$\nabla f(x_k)$ è il gradiente della funzione nel punto corrente.

ALGORITMI PER LA TEORIA DEI NUMERI: METODO DEL GRADIENTE

Punto iniziale
Il punto iniziale x0 influenza il risultato, soprattutto se la funzione non è convessa.
Per funzioni con più minimi locali, partire da punti diversi può portare a risultati diversi.

Learning rate α
Il parametro α è fondamentale.
Se α è troppo piccolo, l'algoritmo converge lentamente.
Se α è troppo grande, l'algoritmo può oscillare o divergere.
Esempio:
α piccolo → passi molto brevi
α corretto → convergenza stabile
α grande → instabilità

ALGORITMI PER LA TEORIA DEI NUMERI: METODO DEL GRADIENTE

Criterio di arresto
L’algoritmo può fermarsi quando:
$||\nabla f(x)|| < \varepsilon$
oppure quando:
$|f(x_{k+1}) - f(x_k)| < \varepsilon$
oppure dopo un numero massimo di iterazioni.

ALGORITMI PER LA TEORIA DEI NUMERI: METODO DEL GRADIENTE

La complessità dipende dal numero di iterazioni e dal costo di calcolo del gradiente.
Se:

- T è il numero di iterazioni;
- C è il costo del calcolo del gradiente;

allora la complessità temporale è:
$O(T \cdot C)$

La complessità spaziale dipende dalla dimensione del vettore x.
Per un vettore di dimensione d, lo spazio richiesto è generalmente: $O(d)$

ALGORITMI PER LA TEORIA DEI NUMERI: METODO DEL GRADIENTE

```
def gradient_descent(grad_f, x0, alpha=0.1, epsilon=1e-6, max_iter=1000):
    x = x0

    for _ in range(max_iter):
        grad = grad_f(x)

        if abs(grad) < epsilon:
            break

        x = x - alpha * grad

    return x


def grad_f(x):
    return 2 * x


minimo = gradient_descent(grad_f, x0=10)
print(minimo)
```

ALGORITMI QUANTISTICI

Gli algoritmi quantistici vanno interpretati come un'estensione del modello classico di calcolo, non come "magia computazionale".

L'idea centrale è che alcuni problemi possono essere risolti più efficientemente sfruttando proprietà fisiche della **meccanica quantistica**, in particolare **sovrapposizione**, **interferenza** ed **entanglement**.

Partiamo dal concetto fondamentale di **qbit**.

Nel calcolo classico l'unità base è il **bit**, che può assumere valore: 0 oppure 1

Nel calcolo quantistico l'unità base è il **qubit**, che può trovarsi in una combinazione lineare di stati:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

ALGORITMI QUANTISTICI

dove α e β sono numeri complessi e vale:

$$|\alpha|^2 + |\beta|^2 = 1$$

Il significato è probabilistico: quando misuriamo il **qubit**, otteniamo 0 con probabilità $|\alpha|^2$ e 1 con probabilità $|\beta|^2$

Attenzione però: non significa che un computer quantistico “provi tutte le soluzioni contemporaneamente” e poi scelga quella giusta. Questa è una semplificazione fuorviante.
Il punto vero è che un algoritmo quantistico deve manipolare le ampiezze in modo che le risposte sbagliate si cancellino e quelle corrette si rafforzino.

ALGORITMI QUANTISTICI: MODELLO COMPUTAZIONALE

Un algoritmo classico è una sequenza di istruzioni su bit.

Un algoritmo quantistico è una sequenza di operazioni unitarie, cioè trasformazioni reversibili applicate a qubit.Alla fine avviene una misurazione, che produce un risultato classico.

Quindi possiamo schematizzare così:

input classico→preparazione quantistica→operazioni unitarie→misurazione

Dal punto di vista del corso di algoritmi, ciò che ci interessa è soprattutto la complessità computazionale: quanti passi richiede un algoritmo quantistico rispetto a uno classico?

ALGORITMI QUANTISTICI: MODELLO COMPUTAZIONALE

Nel calcolo classico avete incontrato classi come P, NP, NP-completo.

Nel calcolo quantistico una classe fondamentale è BQP, cioè: Bounded-error Quantum Polynomial time

È la classe dei problemi risolvibili da un computer quantistico in tempo polinomiale con probabilità di errore limitata.

In modo informale:
P: problemi risolvibili efficientemente da algoritmi classici deterministici.
BPP: problemi risolvibili efficientemente da algoritmi probabilistici classici.
BQP: problemi risolvibili efficientemente da algoritmi quantistici.
NP: problemi le cui soluzioni sono verificabili efficientemente.

ALGORITMI QUANTISTICI: MODELLO COMPUTAZIONALE

È importante chiarire una cosa:

non sappiamo se i computer quantistici risolvano efficientemente tutti i problemi NP-completi. In generale, non si ritiene che un computer quantistico possa risolvere tutti i problemi NP-completi in tempo polinomiale.

Quindi:

Quantistico ≠ Soluzione automatica di NP-completo

ALGORITMI QUANTISTICI: MODELLO COMPUTAZIONALE

È importante chiarire una cosa:

non sappiamo se i computer quantistici risolvano efficientemente tutti i problemi NP-completi. In generale, non si ritiene che un computer quantistico possa risolvere tutti i problemi NP-completi in tempo polinomiale.

Quindi:

Quantistico ≠ Soluzione automatica di NP-completo

ALGORITMI QUANTISTICI: MODELLO COMPUTAZIONALE

Aspetto	Algoritmi classici	Algoritmi quantistici
Unità di informazione	Bit	Qubit
Stato	0 oppure 1	Sovrapposizione di 0 e 1
Operazioni	Porte logiche classiche	Porte quantistiche
Output	Deterministico o probabilistico	Probabilistico dopo misurazione
Modello	Macchina di Turing / RAM	Circuiti quantistici
Vantaggio	General-purpose maturo	Veloce per alcuni problemi specifici
Esempi	Merge sort, Dijkstra, RSA	Shor, Grover, Deutsch-Jozsa

ALGORITMI QUANTISTICI: MODELLO COMPUTAZIONALE

Il confronto più importante riguarda la complessità computazionale.
Un algoritmo quantistico è interessante quando risolve un problema con una complessità asintoticamente migliore rispetto ai migliori algoritmi classici conosciuti.
Esempi:

Problema	Miglior approccio classico noto	Approccio quantistico
Ricerca non strutturata	$O(n)$	$O(\sqrt{n})$ con Grover
Fattorizzazione intera	sub-esponenziale classico	polinomiale con Shor
Logaritmo discreto	difficile classicamente	polinomiale con Shor
Ordinamento	$O(n \log n)$	nessun grande vantaggio generale noto

ALGORITMI QUANTISTICI: ALGORITMO DI SHOR

L’algoritmo di Shor è uno degli algoritmi quantistici più famosi. È stato proposto da Peter Shor nel 1994 ed è importante perché permette di risolvere in tempo polinomiale il problema della fattorizzazione di interi.

Il problema è:
Dato un numero intero N, trovare i suoi fattori primi. Esempio:
N = 15
15 = 3 × 5
oppure:
N = 21
21 = 3 × 7
Per numeri piccoli è facile, ma per numeri con centinaia o migliaia di cifre il problema diventa estremamente difficile per i computer classici.

ALGORITMI QUANTISTICI: ALGORITMO DI SHOR

La sicurezza di molti sistemi crittografici classici, come RSA, si basa sulla difficoltà pratica della fattorizzazione di numeri molto grandi.

RSA usa il fatto che è facile moltiplicare due grandi numeri primi:
$p \times q = N$
ma è difficile fare il contrario:
$N \rightarrow p, q$

L'algoritmo di Shor mostra che, su un computer quantistico sufficientemente potente e affidabile, questa difficoltà potrebbe essere superata.

Quindi Shor ha un enorme impatto teorico sulla crittografia.

ALGORITMI QUANTISTICI: ALGORITMO DI SHOR

Complessità dell'algoritmo di Shor

Classicamente, non si conosce un algoritmo polinomiale efficiente per fattorizzare numeri interi grandi.
L'algoritmo di Shor, invece, risolve la fattorizzazione in tempo polinomiale rispetto al numero di bit di N.
Se n è il numero di bit necessari per rappresentare N, cioè:
$n = \log_2 N$

allora Shor ha complessità polinomiale in n.

In modo semplificato:
O((log N)^k)
per qualche costante k.
Questo è molto più efficiente rispetto agli algoritmi classici noti per la fattorizzazione, che sono sub-esponenziali ma non polinomiali.

ALGORITMI QUANTISTICI: ALGORITMO DI SHOR

Confronto tra fattorizzazione classica e Shor

Aspetto	Algoritmi classici	Algoritmo di Shor
Problema	Fattorizzazione intera	Fattorizzazione intera
Modello di calcolo	Classico	Quantistico
Complessità	sub-esponenziale nota	polinomiale
Risultato	esatto	probabilistico, ma verificabile
Uso principale	Teoria dei numeri, crittografia	Teoria quantistica, crittografia
Impatto su RSA	RSA rimane sicuro classicamente se le chiavi sono grandi	RSA sarebbe vulnerabile con computer quantistici sufficientemente potenti

ALGORITMI QUANTISTICI: ALGORITMO DI SHOR

Per capire meglio il ruolo di Shor, è utile confrontarlo con un altro algoritmo quantistico famoso: Grover.

Grover risolve il problema della ricerca in uno spazio non strutturato.

Se abbiamo n elementi e vogliamo trovare quello corretto:
Algoritmo classico: O(n)
Algoritmo di Grover: O(√n)

Quindi Grover dà un vantaggio quadratico.Shor invece dà un vantaggio molto più forte per la fattorizzazione, perché passa da un problema classicamente molto difficile a un algoritmo quantistico polinomiale.

ALGORITMI QUANTISTICI: ALGORITMO DI SHOR

Algoritmo	Problema	Vantaggio
Grover	Ricerca non strutturata	Quadratico
Shor	Fattorizzazione	Esponenziale rispetto agli algoritmi classici noti
Deutsch-Jozsa	Proprietà di funzioni	Vantaggio teorico
Simon	Periodicità nascosta	Vantaggio esponenziale teorico

ALGORITMI QUANTISTICI: ALGORITMO DI SHOR

```python
import math
import random


def trova_periodo_classico(a, N):
    """
    Trova il periodo r della funzione:
        f(x) = a^x mod N

    Cioè il più piccolo r > 0 tale che:
        a^r ≡ 1 mod N
    """
    valore = 1

    for r in range(1, N):
        valore = (valore * a) % N

        if valore == 1:
            return r

    return None
```

ALGORITMI QUANTISTICI: ALGORITMO DI SHOR

```
def shor_semplificato(N):
    """
    Versione didattica e semplificata di Shor.
    Funziona bene solo per numeri piccoli.
    """

    if N % 2 == 0:
        return 2, N // 2

    while True:
        # 1. Scegli un numero casuale a con 1 < a < N
        a = random.randint(2, N - 2)

        print(f"Scelto a = {a}")

        # 2. Calcola gcd(a, N)
        d = math.gcd(a, N)

        # Se gcd(a, N) > 1, abbiamo già trovato un fattore
        if d > 1:
            return d, N // d

        # 3. Trova il periodo r di a^x mod N
        r = trova_periodo_classico(a, N)
```

ALGORITMI QUANTISTICI: ALGORITMO DI SHOR

```
print(f"Periodo trovato r = {r}")

if r is None:
    continue

# 4. Il periodo deve essere pari
if r % 2 != 0:
    print("r è dispari, riprovo...")
    continue

# 5. Calcola a^(r/2)
x = pow(a, r // 2, N)

# Se x ≡ -1 mod N, il tentativo non è utile
if x == N - 1:
    print("a^(r/2) ≡ -1 mod N, riprovo...")
    continue

# 6. Calcola i fattori usando il massimo comune divisore
fattore1 = math.gcd(x - 1, N)
fattore2 = math.gcd(x + 1, N)

if fattore1 != 1 and fattore2 != 1:
    return fattore1, fattore2
```

ALGORITMI QUANTISTICI: ALGORITMO DI SHOR

```
# Esempio
N = 15
fattori = shor_semplificato(N)

print(f"\nFattori di {N}: {fattori[0]} e {fattori[1]}")
```

ALGORITMI QUANTISTICI: ALGORITMO DI GROVER

```
!pip install qiskit qiskit_aer

from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator

qc = QuantumCircuit(2, 2)

# 1. Sovrapposizione uniforme
qc.h([0, 1])

# 2. Oracolo: marca lo stato |11>
qc.cz(0, 1)

# 3. Diffusore di Grover
qc.h([0, 1])
qc.x([0, 1])
qc.h(1)
qc.cx(0, 1)
qc.h(1)
qc.x([0, 1])
qc.h([0, 1])
```

ALGORITMI QUANTISTICI: ALGORITMO DI GROVER

```
# 4. Misura
qc.measure([0, 1], [0, 1])

sim = AerSimulator()
result = sim.run(qc, shots=1024).result()
counts = result.get_counts()

print(counts)
```

ALGORITMI AD APPRENDIMENTO AUTOMATICO

Gli algoritmi ad apprendimento automatico, o machine learning, sono algoritmi che permettono a un sistema di “imparare” da dati ed esempi, invece di essere programmato esplicitamente con tutte le regole.

In un algoritmo classico spesso scriviamo direttamente le istruzioni:

input → regole scritte dal programmatore → output

Nel machine learning, invece, forniamo dati e il sistema costruisce un modello:

dati → apprendimento → modello → previsioni

L’obiettivo è trovare regolarità nei dati e usarle per fare predizioni, classificazioni o decisioni.

ALGORITMI AD APPRENDIMENTO AUTOMATICO

Un algoritmo di apprendimento automatico riceve un insieme di dati, chiamato dataset.

Ogni elemento del dataset può essere descritto da alcune caratteristiche, dette feature.

Esempio: vogliamo prevedere il prezzo di una casa.

Le feature possono essere:

- Superficie
- numero di stanze
- Zona
- anno di costruzione
- distanza dal centro

ALGORITMI AD APPRENDIMENTO AUTOMATICO

Il valore da prevedere, per esempio il prezzo, si chiama target.

L'algoritmo cerca una funzione che colleghi input e output:
feature → target

In forma generale:
$y = f(x)$

dove:
x rappresenta i dati in input;
y rappresenta l'output desiderato;
f è il modello appreso.

ALGORITMI AD APPRENDIMENTO AUTOMATICO: FASI PRINCIPALI

1. Raccolta dei dati. I dati sono fondamentali. Un modello non può imparare bene se i dati sono pochi, rumorosi o sbilanciati. Esempio: per classificare email spam, servono molte email già etichettate.

2. Pulizia e preparazione. Prima dell'apprendimento, i dati devono essere preparati. Operazioni comuni:
   - rimozione di valori mancanti
   - normalizzazione
   - codifica di variabili categoriche
   - eliminazione di duplicati
   - bilanciamento delle classi

Questa fase è molto importante perché influisce direttamente sulla qualità del modello.

ALGORITMI AD APPRENDIMENTO AUTOMATICO: FASI PRINCIPALI

3. Suddivisione del dataset. Di solito si divide il dataset in:

- training set
- test set

Il training set serve per addestrare il modello.
Il test set serve per valutare il modello su dati mai visti prima.
Spesso si usa anche un validation set per scegliere i parametri migliori

4. Addestramento

Durante l'addestramento, il modello modifica i propri parametri per ridurre l'errore.
Si definisce una funzione di errore, detta loss function.
Esempio:
errore = previsione - valore reale
Il modello cerca di minimizzare questa perdita.
Molti algoritmi usano il metodo del gradiente per aggiornare i parametri

ALGORITMI AD APPRENDIMENTO AUTOMATICO: FASI PRINCIPALI

5. Valutazione. Dopo l'addestramento, il modello viene testato su dati non usati durante il training.

Metriche comuni:

Per classificazione:
accuracy
precision
recall
F1-score
matrice di confusione

Per regressione:
MSE
MAE
RMSE
R^2

ALGORITMI AD APPRENDIMENTO AUTOMATICO: FASI PRINCIPALI

Le principali categorie sono:

- Apprendimento supervisionato
- Apprendimento non supervisionato
- Apprendimento per rinforzo

ALGORITMI AD APPRENDIMENTO AUTOMATICO: FASI PRINCIPALI

Algoritmo	Tipo	Uso principale	Vantaggi	Limiti
Regressione lineare	Supervisionato	Regressione	Semplice, interpretabile	Solo relazioni lineari
Regressione logistica	Supervisionato	Classificazione	Probabilistica, semplice	Confini lineari
KNN	Supervisionato	Classificazione/regressione	Intuitivo	Lento in predizione
Albero decisionale	Supervisionato	Classificazione/regressione	Interpretabile	Overfitting
Random Forest	Supervisionato	Classificazione/regressione	Robusto	Meno interpretabile
SVM	Supervisionato	Classificazione	Margine massimo	Costoso su grandi dataset
K-means	Non supervisionato	Clustering	Veloce	Serve scegliere k
Reti neurali	Supervisionato/non supervisionato	Dati complessi	Molto potenti	Costose e poco interpretabili

ALGORITMI AD APPRENDIMENTO AUTOMATICO: FASI PRINCIPALI

```
from sklearn.linear_model import LinearRegression
import numpy as np

# Dati di esempio:
# x = superficie della casa in metri quadrati
# y = prezzo in migliaia di euro

X = np.array([[50], [60], [80], [100], [120]])
y = np.array([150, 180, 240, 300, 360])

# Creazione del modello
modello = LinearRegression()

# Addestramento
modello.fit(X, y)

# Predizione
nuova_casa = np.array([[90]])
prezzo_previsto = modello.predict(nuova_casa)

print("Prezzo previsto:", prezzo_previsto[0], "mila
euro")
```

ALGORITMI AD APPRENDIMENTO AUTOMATICO: FASI PRINCIPALI

```
import random
import math

def distanza(a, b):
    return math.sqrt(sum((x - y) ** 2 for x, y in zip(a, b)))

def media_punti(punti):
    dimensione = len(punti[0])
    return [
        sum(p[i] for p in punti) / len(punti)
        for i in range(dimensione)
    ]

def k_means(punti, k, iterazioni=10):
    centroidi = random.sample(punti, k)
```

ALGORITMI AD APPRENDIMENTO AUTOMATICO: FASI PRINCIPALI

```
    for _ in range(iterazioni):
        cluster = [[] for _ in range(k)]

        for punto in punti:
            indice_vicino = min(
                range(k),
                key=lambda i: distanza(punto, centroidi[i])
            )
            cluster[indice_vicino].append(punto)

        for i in range(k):
            if cluster[i]:
                centroidi[i] = media_punti(cluster[i])

    return centroidi, cluster


punti = [
    [1, 2], [2, 1], [1, 1],
    [8, 9], [9, 8], [8, 8]
]

centroidi, cluster = k_means(punti, k=2)

print("Centroidi:", centroidi)
print("Cluster:", cluster)
```

ANALISI DEGLI ALGORITMI

Referenze

- Algoritmi e Strutture Dati – Demetrescu, Finocchi, Italiano
- Introduzione agli algoritmi e strutture dati – Cormen, Leiserson, Rivest, Stein

Risorse Python

https://runestone.academy/ns/books/published/pythonds/index.html?utm_source=chatgpt.com

https://github.com/0bprashanthc/algorithm-books/blob/master/Data%20Structures%20%26%20Algorithms%20in%20Python.pdf?utm_source=chatgpt.com

https://edu.anarcho-copy.org/Programming%20Languages/Python/Python%20Data%20Structures%20and%20Algorithms.pdf?utm_source=chatgpt.com

https://do1.dr-chuck.com/pythonlearn/IT_it/pythonlearn.pdf?utm_source=chatgpt.com

